

[Sign up](#)[Home](#) > [QuillBot Tools](#) >[Compressing Large Language Generation Models with Sequence-Level Knowledge Distillation](#)

Compressing Large Language Generation Models with Sequence-Level Knowledge Distillation

Published on June 26, 2021 by [Paige Pfeifer, BA](#) Revised on November 24, 2025

By Brendan Chambers, David Silin, and Kevin Gimpel of QuillBot Research

It's important to consider cultural nuances when communicating with stakeholders from different backgrounds.

Understanding these subtleties fosters better collaboration and enriches the creative process itself. More diverse perspectives should not be seen as introducing friction to the decision-making, but on the contrary, they help reach more well-considered more solutions and anticipate future challenges and oversights.

Free AI Detector

☰ Table of contents

1. Brief recap of big changes in language modeling
2. A snippet of our research process
3. Highlighting prior work

4. Experiments
5. Results
6. Discussion

Brief recap of big changes in language modeling

We are a little obsessed with language generation and deep transfer learning here. You have probably heard murmurs about the breakthroughs. It has really been a thrilling five years following research progress in machine learning and natural language processing. Building on deep foundations from [machine translation](#), computer vision, and NLP itself, the research community passed through an inflection point and rapidly developed new possibilities for engineering statistical intelligence. As a result, we are seeing big leaps in applications involving sequential data: from automatic translation systems to semantic search, description of visual scenes, multilingual speech transcription, music synthesis, and more. All these hard problems are far from solved, but breakthroughs in language modeling are advancing the frontier of possibility and reshaping our experiences of human computer interaction, in ways which are perhaps only beginning to be realized.



Let's step back. Language modeling is a family of statistical procedures for predicting missing words from their context, often in a left-to-right direction. Humans are great at this. Try it yourself:



The first planned industrial park in the United States was located on the South Side of Chicago, at a strategic ____ accessible to transportation by ship and rail.



You probably filled in *location* or *site* or some other probable candidate word. Recently, deep learning systems have become really good at language modeling, too (1). There have been some big advances in the design of model architectures and decoding methods (2, 3) which are sensitive to contextual semantics and built to run fast on modern hardware. We have also seen incredible refinement of the transfer learning paradigm (4, 5).

In transfer learning, model parameters typically get most of their updates without traditional labeled data, leveraging self-supervised signals such as missing-word prediction (*masked language modeling*) to learn from billions of words sampled from numerous text domains. Then, these pretrained models get re-adapted for downstream tasks, finetuned on smaller curated datasets where we do have target annotations available. Practical transfer learning

was a huge breakthrough for natural language processing, because annotated training sets are scarce, but natural text is abundant. Sometimes this procedure is repeated more than once, and there is an art to designing recipes for effective transfer.

Together, these advances enabled huge leaps in scale (6), and we now have extraordinarily powerful language models and language generation systems with billions of parameters, like *GPT3*, *T5*, *M2M*, and *DeBERTa* (7, 8, 9, 10). Not only do these giant models have more expansive training capacity, but their overparameterization seems to help them learn more efficiently and generalize more smoothly to unseen data.

At the same time, the research community is working to understand when these models are safe to use, because deep learning systems can echo back serious human prejudices (11). When models are used to make algorithmic judgments that impact human lives, they can deepen existing inequalities of power. In addition, training titanic learning systems imposes high carbon costs, further straining global ecologies. So we at once celebrate this incredible progress, while also taking a critical view towards technological utopias. Ultimately we design our tools for *humans*: to sift and revise the semantic maps learned by language models, with the judgement and distinct voice of a human in the loop.

Language models are at the core of the clever, contextually aware assistive writing tools we offer at QuillBot (which are also carefully finetuned on human-curated internal data, and interwoven with additional components and secret ingredients).

But there is a fundamental contradiction for serving these state-of-the-art models in the real world: they are just too big. Billion-parameter scale models need advanced hardware to run, but this hardware isn't available at the scale of millions of users per day. In addition, massive-scale models impose an outsized strain on the global ecology, with renewable energy infrastructure limited in capacity and low-power electronics not yet widespread. Processing latency is a third serious obstacle, because round trip delays can stretch into multiple seconds and beyond. For all these reasons, the unit economics for delivering moon-sized models to users are unfavorable. And that brings us to the subject of this post: a glimpse at how we gently coax "titanic, truly enormous" models down to merely "very large" sizes through teacher-student compression methods. The procedure we will talk about below is known as sequence level knowledge distillation.



A snippet of our research process

At QuillBot, we sometimes screen train recipes using automatic measures of quality. In this post, we are sharing measurements of English to French translation BLEU scores, across a range of model sizes and finetuning recipes. Translation pairs are taken from a slice of ParaCrawl English-French data composed using some of our internal filters. In this case we are presenting a distillation recipe for a 48.7x reduction in parameter count, and assessing its cost by comparing output [translations](#) to a set of references under the measure known as BLEU.

Highlighting prior work

It was not initially clear whether frameworks for knowledge distillation would generalize from vision systems and encoder-only models to the setting of encoder-decoder architectures. For models with decoders, teacher likelihoods at a single position might interact in unexpected ways with aggregate predictions obtained through sequence decoding heuristics. Similarly, exposure bias separating training through teacher forcing versus sequential prediction at inference time might pose challenges to distillation.

These concerns were resolved empirically by [Kim and Rush \(2016\)](#), who demonstrated that word-level distillation (based on cross-entropy over next-token distributions from the teacher model) remains effective even in encoder-decoder models. The authors then present a satisfying sequence-level approach which is even stronger: output sequences generated by a teacher model are simply paired with their respective input sequences and supplied to a student model as pseudolabeled training data. Kim and Rush show that the two methods are synergistic, and filtering candidate teacher outputs helps even more. Below, we focused our tests on the sequence-level methodology.

The second key reference we will highlight is [Shleifer and Rush \(2020\)](#), reporting experiments in compressing [summarization models](#), e.g. *DistillBART*. These experiments echo other recent work in distillation, leveraging selective initialization of layers and using additional losses to align internal features. But in the setting of English to Romanian translation, sequence-level distillation using pseudolabels actually out-performed these more extensively engineered recipes. Of course the two approaches are likely complementary and could be used in combination. Overall, perhaps because sequence-level approaches lessen the mismatch in



train/inference exposure, Shleifer and Rush provide additional evidence that encoder-decoder models can be effectively compressed using teacher output sequences.

Experiments

The references above provide strong guidance for designing sequence-level distillation recipes, but in any particular application, a number of design choices remain. We set out to measure whether our implementations and extensions of these distillation recipes would be equally effective, and we share some of these results below.

We hypothesize that the problem of translation is closely related to monolingual paraphrasing, which both necessitate close semantic alignment across model inputs and outputs. English to French translation was included in the pretraining regimen for the original models via *WMT2014* data. We imposed a domain shift relative to the original pretraining data, using a selection from *ParaCrawl En-Fr* screened using in-house filters and sampled down to 300k pairs. A second training set was obtained by further subsampling the data to 30k pairs, to test the impact of training data abundance.

Performance was assessed with the BLEU score, using the *SacreBLEU* implementation. The test set was composed of 1000 labeled pairs, partitioned into five splits to monitor test variability. We carried out the following experiments:

(1) *baseline translation BLEU*

(2) *translation BLEU after finetuning*

(3) *translation BLEU after 50x parameter compression through distillation*

Results

To measure baseline performance, we evaluated four pretrained *T5* models on filtered English to French bitext. The models are not naive to this task: in addition to other varied data sources, the models were originally pretrained on English to French translation using the *WMT 2015 EnFr* dataset. Baseline performance on a filtered *ParaCrawl EnFr* slice is relatively strong, even though the data distribution is somewhat different from the translation data seen during pretraining. Overall, performance increases with model size, with the largest improvement occurring at the transition from the *small* to *base* model sizes



(from 60 million to 220 million parameters). The two larger models we assess here are made up of 770 million and 2.8 billion parameters, respectively. Before finetuning, these *large* and *3b* sizes performed similarly, hinting at unused capacity in the pretrained 3b model. Next, we measure improvement in performance after finetuning the smallest and largest models (*small* and *3b*).

After finetuning on a sample of thirty thousand translation pairs (*30k*), the performance of the smallest model improved dramatically. This small model nearly matched the upper limit of the naive baselines set by a model with approximately 50x more parameters. The largest model also saw significant improvement, even with the limited number of training pairs.

We repeated these experiments using 10x more training data, on three hundred thousand translation pairs (*300k*). Notably, the performance of the data rich small model (*sm 300k*) exceeded the performance of the data limited giant model (*3b 30k*). However, the data rich giant model (*3b 300k*) was strongest of all by a significant margin. Next, using this *3b 300k* model as a teacher, we compare two sequence-level distillation recipes for compressing it to fewer parameters.

We selected the source sentences from the 300k data slice, and used them to generate 300k new training pairs, with new pseudo-target sentences generated from the greedy outputs of the teacher model. Because it is generated following the sequence-level knowledge distillation recipe, we refer to this data as *seq* in these results. We next verify whether the *seq* teacher data can effectively improve student performance in a small finetuned model with approximately 50x fewer parameters.

We compared two recipes for leveraging the *seq* pseudolabels. In the first distillation experiment, we began with a small pretrained model, finetuned on 30k pairs from the original dataset, and then finetuned again using the 300k *seq* teacher outputs. That is, we took the model tested in the first finetuning experiment *sm 30k* and attempted to improve it using pseudolabels from the stronger teacher model *3b 300k*. In this experiment, the small model did not improve under teacher instruction: in fact we measured a slight decrease in translation BLEU relative to the finetune-only *sm 30k* model. We also observed a decrease in variability over the evaluation splits.



In the second distillation experiment, we began with the same small pretrained model, finetuned on the 300k *seq* teacher outputs, and only then finetuned on the original 30k labeled pairs. This approach was very successful: the small model improved precipitously, outperforming all other recipes we tested here and nearly matching the performance of its gigantic teacher.

Discussion

The work described in this post is a slice from our wider projects around evaluation and distillation of large language models. When it comes to choosing the right model, and the right data, and the right training recipe, benchmarking experiments guide us and help us make informed choices. However, ultimately it is the experience of users, situated in the context of their habits and preferences, that defines whether technology work is successful. So we caution that automatic metrics of quality are limited, that translation is only a proxy for our core assistive writing tools, that domain shifts between training and deployment introduce new failure modes. These complications and entangled design choices are one reason deep learning is so engaging to practice. Overall our results spell out a simple and strong recipe for distilling huge encoder-decoder models for language generation, and we achieve a 48.7x reduction in parameters at the cost of 0.7 BLEU. Recipes like this one enable us to deliver smarter tools that help millions of users write better and faster.



Further reading



Introduction to language modeling with deep transformer networks:

1. [\[1810.04805\] BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#)
2. [Attention is All you Need](#)
3. [\[1904.09751\] The Curious Case of Neural Text Degeneration](#)
4. [\[1801.06146\] Universal Language Model Fine-tuning for Text Classification](#)
5. [\[2004.10964\] Don't Stop Pretraining: Adapt Language Models to Domains and Tasks](#)
6. [\[2001.08361\] Scaling Laws for Neural Language Models](#)
7. [\[2005.14165\] Language Models are Few-Shot Learners](#)
8. <https://jmlr.org/papers/volume21/20-074/20-074.pdf>
9. https://github.com/pytorch/fairseq/tree/master/examples/m2m_100

10. DEBERTA: DECODING-ENHANCED BERT WITH DISENTANGLED ATTENTION

11. On the Dangers of Stochastic Parrots | Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency

Knowledge distillation in natural language generation:

12. Sequence-Level Knowledge Distillation – ACL Anthology

13. [1503.02531] Distilling the Knowledge in a Neural Network

14. DistilBERT: <https://arxiv.org/pdf/1910.01108.pdf>

15. LayerDrop – <https://openreview.net/forum?id=SylO2yStDr>

16. [2010.13002] Pre-trained Summarization Distillation

17. Ensemble distillation: <https://www.aclweb.org/anthology/2020.emnlp-main.450.pdf>

18. TinyBERT: <https://www.aclweb.org/anthology/2020.findings-emnlp.372.pdf>

19. Movement pruning: <https://arxiv.org/abs/2005.07683>

20. Block sparse layers: https://huggingface.co/blog/pytorch_block_sparse

21. Bert of Theseus: <https://www.aclweb.org/anthology/2020.emnlp-main.633.pdf>

22. Tangled up in BLEU: Reevaluating the Evaluation of Automatic Machine Translation Evaluation Metrics

23. Understanding knowledge distillation in non-autoregressive machine translation: <https://arxiv.org/pdf/1911.02727.pdf>

24. Self-distillation amplifies regularization in Hilbert Space: <https://arxiv.org/abs/2002.05715>

25. Self-knowledge generation: a simple way for better generation: <https://arxiv.org/pdf/2006.12000.pdf>

